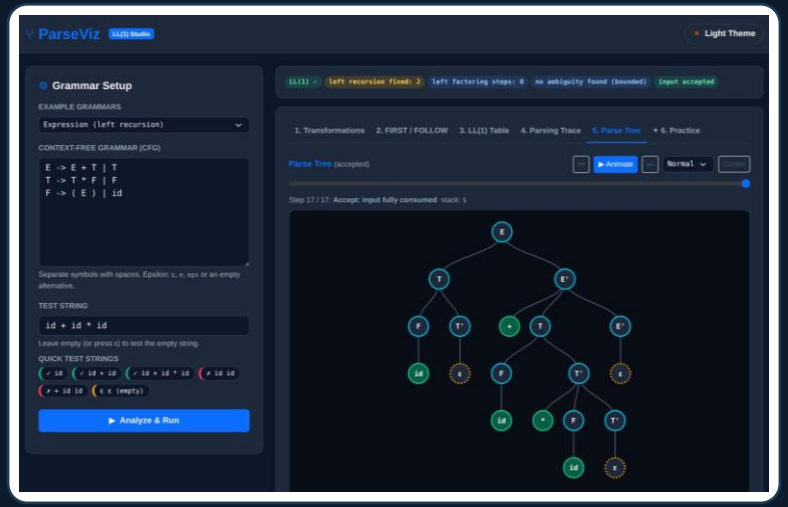
ParseViz

Interactive LL(1) Parsing Studio

PS-03 · Unit II: Top-Down Parsing
CFG, left recursion, left factoring, FIRST/FOLLOW, LL(1)



Why parsing feels abstract

1

Steps are memorised

Students learn the recipe for removing left recursion without knowing why it is needed.

2

Work stays hidden

FIRST/FOLLOW sets and table cells appear as answers, not as derivations.

3

No quick feedback

A rejected string rarely tells you where or why the parse failed.

ParseViz shows every transformation, set, table cell, stack step and tree node, then lets students test themselves.

One pipeline, six views

1. Grammar
2. Left recursion
3. Left factoring
4. FIRST / FOLLOW
5. LL(1) table
6. Trace + tree

Python engine

parser_engine.py: every algorithm, no web code

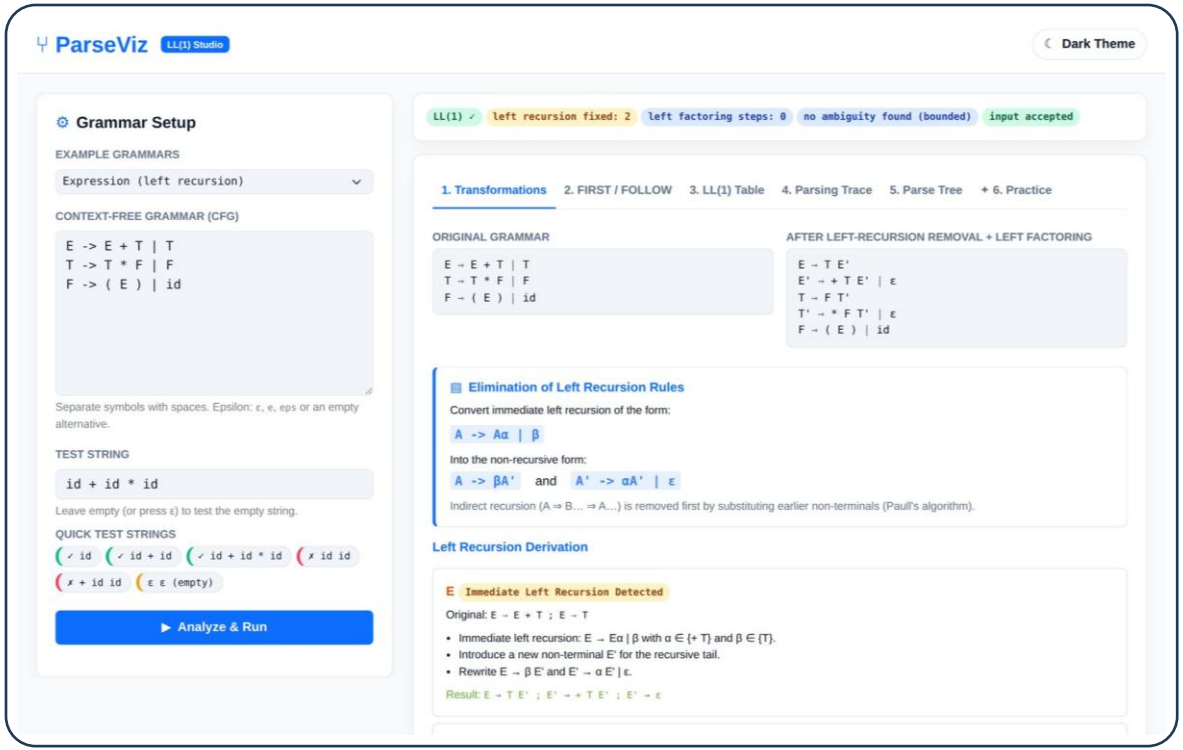
Flask API

/api/analyze, /api/practice, /api/practice/check

Web front end

Bootstrap tabs + D3 tree, synced to one step counter

The studio at a glance



Fixed setup panel

Grammar and test string stay in view while results scroll

Pinned tab bar

Jump between the six stages without scrolling back up

Live status chips

LL(1) verdict, fixes applied and accept or reject at a glance

Removing left recursion

Rule

```
A → A α | β
⇒  A → β A'
     A' → α A' | ε
```

Left Recursion Derivation

E **Immediate Left Recursion Detected**

Original: E → E + T ; E → T

- Immediate left recursion: E → Eα | β with α ∈ {+ T} and β ∈ {T}.
- Introduce a new non-terminal E' for the recursive tail.
- Rewrite E → β E' and E' → α E' | ε.

Result: E → T E' ; E' → + T E' ; E' → ε

What the studio handles

- α and β shown for each non-terminal
- A → A | ε gives A → A' (no stray ε)
- Indirect recursion removed first by substitution (Paull)
- Warning when a rule has no non-recursive alternative
- Trace stops after 3000 steps instead of looping forever

Left factoring

Before

```
A → a b c | a b d | a x | y
```

Left Factoring Derivation

A **Factored** common prefix: a

Before: `A → a b c ; A → a b d ; A → a x ; A → y`

- Productions A → a b c, A → a b d, A → a x share the prefix a (α).
- Pull α out: A → α A' and put the differing tails (β's) under A': b c | b d | x.

After: `A → a A' ; A → y ; A' → b c ; A' → b d ; A' → x`

How it works

- Group productions by first symbol
- Take the longest common prefix α
- Move the differing tails (β) under a new non-terminal
- Repeat on the new rule until no prefix is left
- Each step lists the rules before and after

FIRST and FOLLOW, step by step

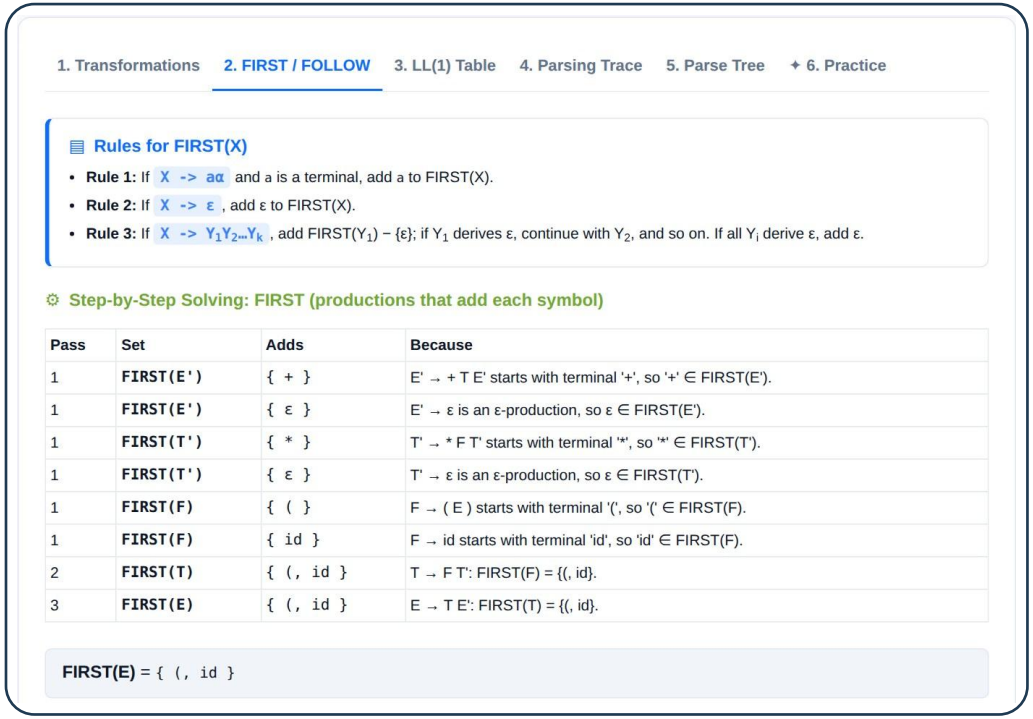
Non-terminal	FIRST	FOLLOW
E	{ (, id }	{) , $ }
E'	{ + , ε }	{) , $ }
T	{ (, id }	{) , + , $ }
T'	{ * , ε }	{) , + , $ }
F	{ (, id }	{) , * , + , $ }

Expression grammar after transformation

Every set is explained

- Logged per pass, per production
- FOLLOW rules 1, 2, 3 tagged on each addition
- Hover a symbol to see the productions behind it
- ε and nullable chains handled

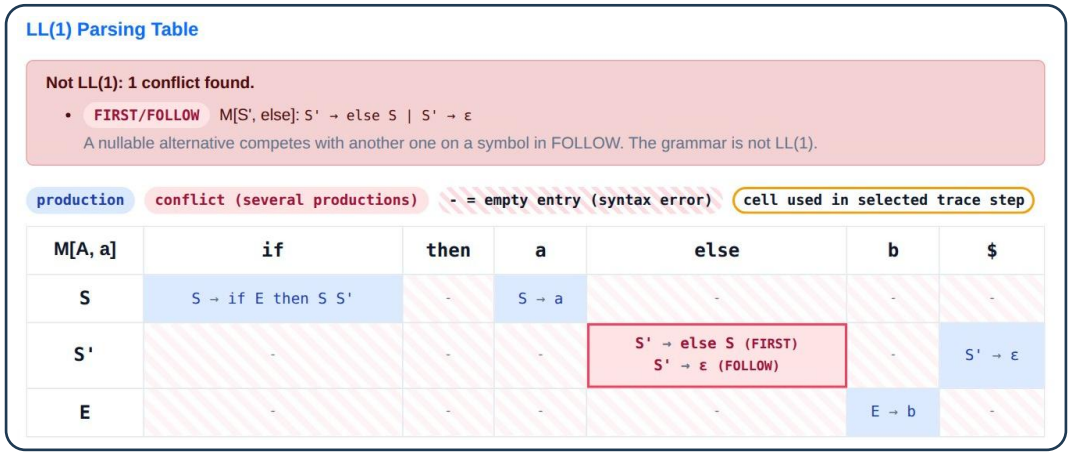
Inside the FIRST / FOLLOW tab



How to read the log

- Rules for FIRST and FOLLOW come first
- One row per addition: pass, set, symbols, reason
- Each reason names the production behind it
- Final sets are listed per non-terminal

LL(1) table and conflicts



If-else grammar: M[S', else] holds two productions

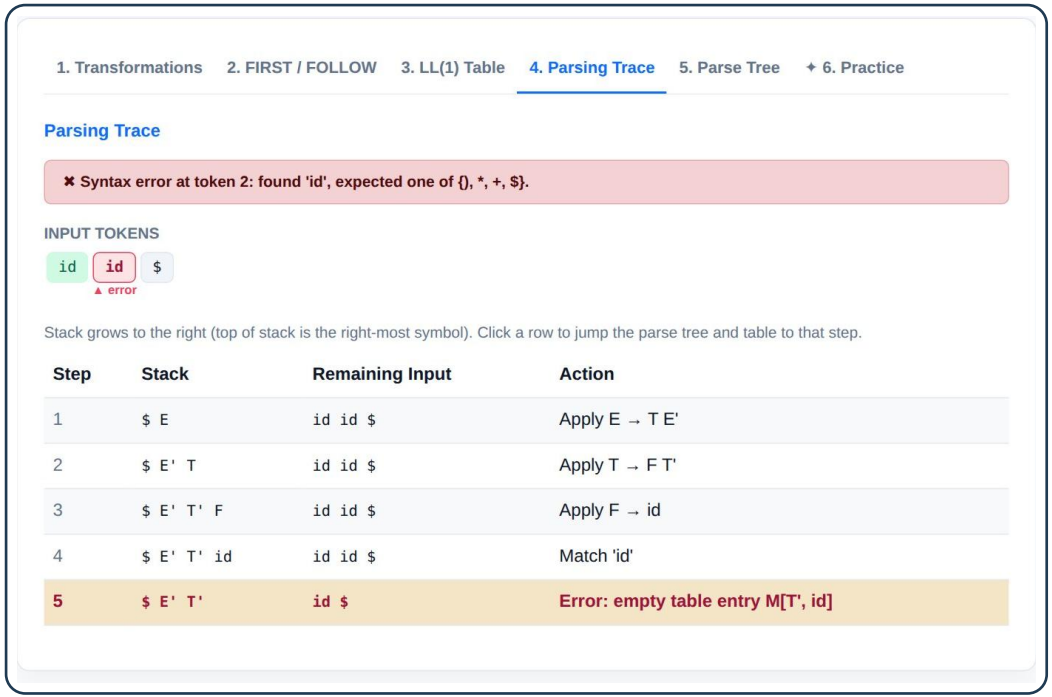
FIRST/FIRST
common prefix or ambiguity

FIRST/FOLLOW
nullable rule competes

FOLLOW/FOLLOW
two nullable rules

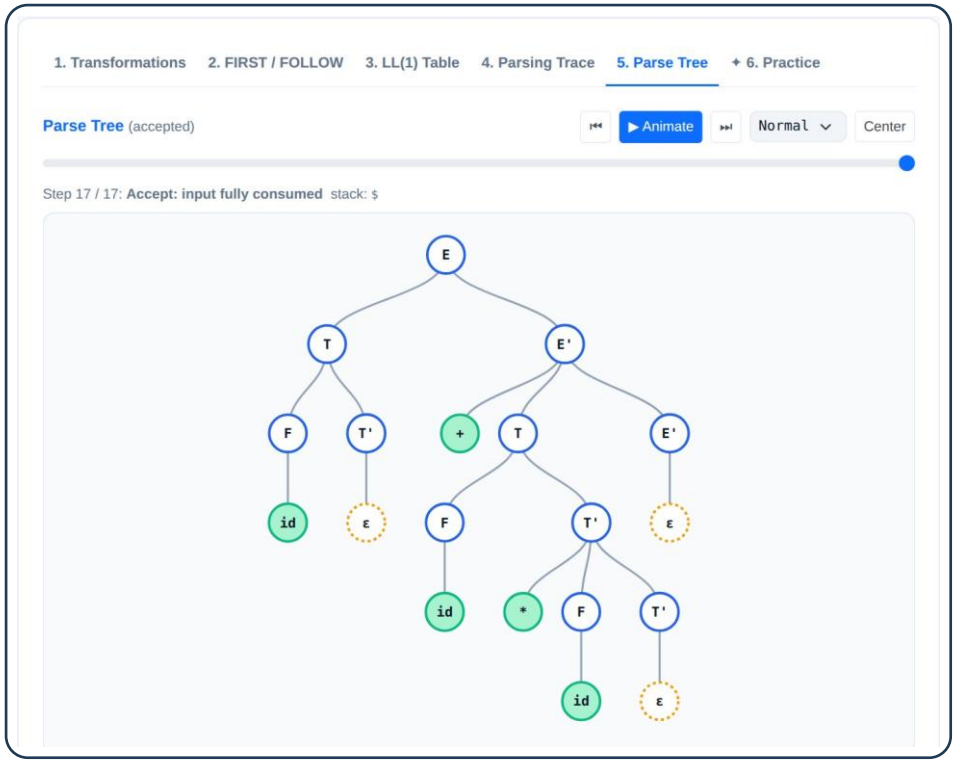
Duplicates removed, all conflicts listed with a fix hint. Empty cells show “-” (syntax error).

Trace with exact error location



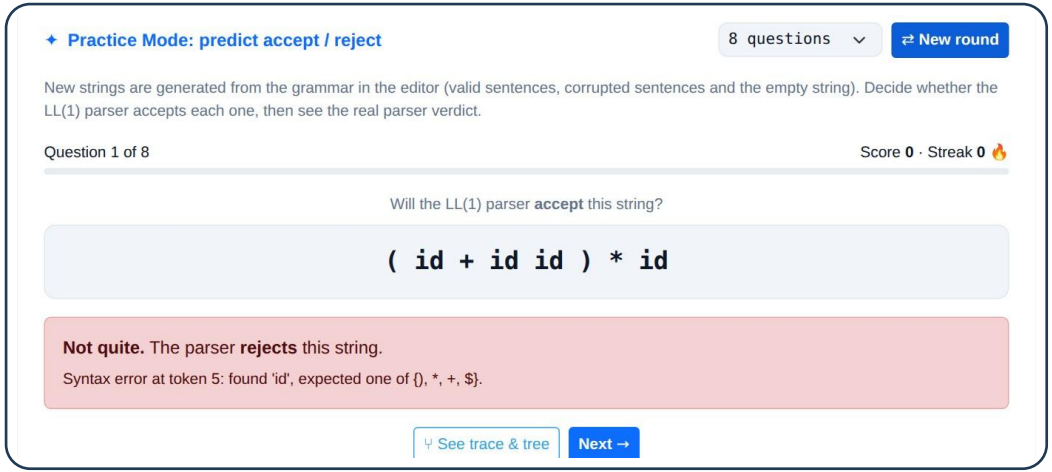
- Stack, remaining input and action at every step
- Rejected input: the bad token is marked, with its position and the expected symbols
- Clicking a row jumps the table and tree to that step
- Empty string and “id+id” without spaces both work

Animated parse tree



- Built from the trace: play, pause, step or drag the slider
- Matched terminals turn green; ε leaves are dashed
- Rejected input shows the partial tree, failing node in red
- Layout is fixed, so nodes fade in without jumping

Practice mode



Bonus challenge: predict accept or reject

Mix of valid sentences, corrupted ones and the empty string.

1. Generate strings from the current grammar
2. Student predicts accept or reject
3. Real parser gives the verdict and the reason
4. Score, streak and review table

Stack and deployment

Python 3 + Flask

engine and JSON API

Bootstrap 5 + D3 v7

tabs, tables, animated tree

gunicorn + Render

public link from GitHub

Source code

`github.com/sphoorthy282007/ParseViz-Interactive-Parse-Tree-Generator`

Run locally: pip install -r requirements.txt, then python app.py

Thank You!

ParseViz — Interactive Parse Tree Studio